PROJECT MILESTONES

Retrieval and Ranking Architectures

CP4285 Modern Recommendation Systems — Week 06

CP4285 Instruction Team

15 Sep 2026

Scan for the slides
soc-n.us/cp4285-t2610-w06

01 Announcements and Context

CHANGELOG

- **24 Aug 2026** — Refreshed the course slide deck and changelog.



Bandit Time: Start TempoQuiz

TEMPOQUIZ HOST LAUNCH

Open the room before Question 1

Open the host dashboard, then ask students to join with the **live QR code or room code shown there**.

LIVE QR + ROOM CODE

Use the host dashboard — do not add a static code here.

HOST DASHBOARD CUE

Week 05 Recap

ROOM
Open
students can join

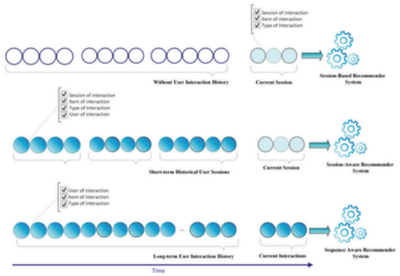
QUESTION
1
ready to release

▶ Release question 1

Wait for the room to settle, then release the first recap question.

WEEK 05 RECAP

Which interaction history is available?



Session-based

Use the ordered events in the current session when no history is available.

Session-aware

Combine the current session with earlier sessions from an identified user.

Sequence-aware

Model a longer ordered history when earlier interactions may still matter.

Source: Ravanmehr, R., & Mohamadrezaei, R. (2024). *Session-based recommender systems using deep learning* (Fig. 1.3, p. 15). Springer. https://doi.org/10.1007/978-3-031-42559-2

👥 Deciding what starts a new session? 📶

Your group owns one session-boundary method. Apply it to the Ask Maps trace, then compare it with the shared 30-minute inactivity baseline.

Shared simplified implementation

Fixed inactivity: begin a new session after 30 minutes with no event. It is clear and cheap, but one threshold cannot fit every task.

1. Adaptive timeout

Change the gap using event pace, device, time of day, or product context.

2. Task similarity

Split when queries, items, or actions indicate a different task.

3. Boundary classifier

Learn a same-task or new-task decision from labelled traces.

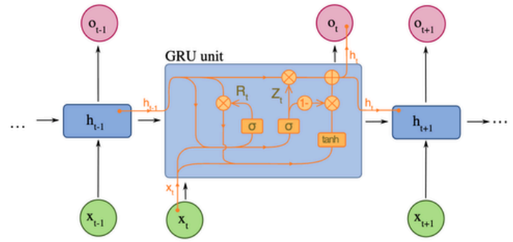
4. Intent change point

Infer when the likely hidden intent state changes.

03:00

▶ Start Timer

GRU4Rec: carry a changing session state



Click diagram for source · fdeloche, *Gated Recurrent Unit*, CC BY-SA 4.0, via Wikimedia Commons.

Update gate z_t

Call out the final blend: how much of the previous hidden state should persist versus be replaced by the candidate state?

Reset gate r_t

Call out the path into the candidate state: how much earlier context should shape the next proposal?

Read the diagram

Both gates receive the current input and previous state. Their outputs shape the candidate and the final blend.

SASRec walkthrough: an older action can matter most

Causal self-attention over the available session prefix

late-night halal food	museum ticket	delivery-time check
state at $t-2$	state at $t-1$	current state t

Step 1 · attention scores

0.72 0.06 0.22

Step 2 · causal softmax weights

72% 6% 22%

Step 3 · contextual state

The state at $t-2$ has the largest weight: it identifies the restaurant-related intent behind the current delivery-time check.

Ranked next-item recommendations

Session prefix
late-night halal food ($t-2$)
→ museum ticket ($t-1$)
→ delivery-time check (t)

1. halal restaurant details
highest-ranked next item

2. nearby halal delivery options
re-rank if the user opens the details

3. order tracking
re-rank after a later observed action

Only past actions are visible. Older actions may matter most.

LAST WEEK PRE-LECTURE EXERCISE

When should the current session outweigh history?

> **A user’s long-term history is vegetarian cafés. In their current session, they search “late-night halal food”, open nearby restaurant pages, and check delivery times. What should the app consider now—and what should it not assume?**

Canvas: 120–140 words by Mon, 14 Sep 2026, 23:59 SGT. Name two session-relevant item types, one risk of overwriting durable preference, and one non-click helpfulness signal.

Next week: retrieve plausible options before ranking and re-ranking them.



NEXT CHECKPOINT

Project Design Critique Update

Show the team’s **scope**, proposed **method**, progress toward the goal, and any preliminary results.

Open the critique template ↗



WHERE IT IS HEADING

STePS portrait poster · 11 Nov

The poster should carry forward the critique's core components. Be ready for a short 5–10 minute conversation about the work; STePS teams submit the poster PDF in Canvas.

Open the poster template ↗



LAST WEEK PRE-LECTURE EXERCISE · CANVAS VOICES

Three Canvas views on session and history

Week 6: Retrieval and Ranking Architectures

WEEK 06 · RETRIEVAL AND RANKING SYSTEMS

Preserve useful evidence early, then spend expensive computation only where it can change the final recommendation.

SYSTEM REASONING

Trace the cascade

Explain the distinct jobs of retrieval, ranking, re-ranking, and context assembly.

COMPLEXITY REASONING

Budget the work

Reason about runtime, memory, candidate depth, embedding dimension, and token budget.

ARCHITECTURE REASONING

Design the hand-offs

Choose what an index, a lightweight ranker, and a deep re-ranker should each know.

STAKEHOLDER REASONING

Audit what is lost

Evaluate recall, provenance, visibility, and source diversity—not only a final relevance score.

02 Pipeline Design



Maya’s current intent: a five-day trip

SHOPPING SESSION

Maya’s long-term purchases are mostly electronics and stationery.

Today she searches for luggage, opens cabin-bag dimensions, compares wheels and weight, and checks delivery before Friday.

CURRENT QUERY

“compact carry-on luggage for a five-day trip”

FICTIONAL PRODUCT CATALOGUE

N ≈ 5,000,000 active SKUs

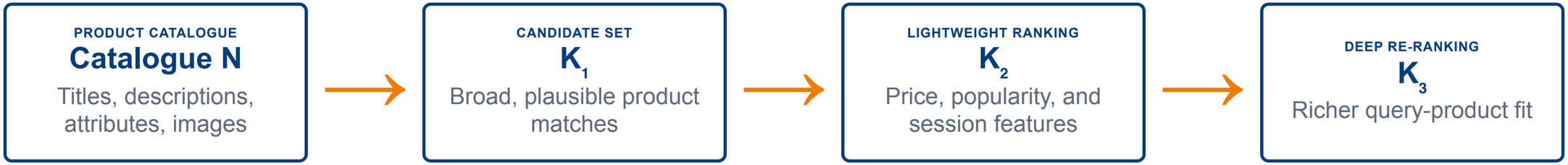
Illustrative catalogue records—not retrieved candidates.

SKU-LUG-204 Compact Carry-on Spinner · strong overlap	**SKU-BPK-619** 40L Cabin Backpack · strong overlap
SKU-LUG-118 Hard-shell Carry-on · strong overlap	**SKU-CHK-841** 28-inch Checked Suitcase · too large
SKU-TOT-730 Travel Tote · travel, not luggage	**SKU-ORG-501** Packing Cubes · accessory
SKU-PAS-338 Passport Wallet · accessory	**SKU-BPK-280** Laptop Backpack · nearby category

Question: before selecting Maya’s final list, which products must enter the candidate set?

Why candidate generation cannot choose final products

Each stage receives fewer products and is allowed to use richer query and session evidence. A product removed early cannot be recovered later.



Then present: remove near-duplicates, explain the ordering, and return a manageable product shortlist for Maya.

Why not use one deep query-product scoring stage?

SINGLE-STAGE DEEP SCORING

Apply joint attention to every candidate product

Input: K_1 products, each with a query-product sequence of length L.

APPROXIMATE ATTENTION COMPUTE

$$O(K_1 L^2 d)$$

K_1 products × quadratic query-product interaction × hidden dimension d

With $K_1 = 1{,}000$, every request jointly processes 1,000 product descriptions with Maya's query and session. The most expensive model is spent on many weak matches.

STAGED PRODUCT RANKING

Use deep query-product scoring only after cheaper cuts

Generate: $N \rightarrow K_1$ with a lexical, dense, or hybrid candidate index.

Rank: $K_1 \rightarrow K_2$ with lightweight product and session features.

Re-rank: $K_2 \rightarrow K_3$ with joint query-product scoring, $O(K_2 L^2 d)$.

DEEP-SCORING REDUCTION

$$K_1 / K_2$$

$1{,}000 \rightarrow 20$ means 50× fewer query-product evaluations

What must be ready before Maya shops?

BEFORE THE QUERY · OFFLINE

Prepare the product catalogue

- store product title, description, category, and attributes;
- attach price, availability, and stable product identifiers;
- build and cache searchable representations.

Output: an indexed product catalogue with stable IDs.

WHEN MAYA SEARCHES · ONLINE

Respond to her query and current session

- encode the query and current-session signals;
- retrieve plausible product candidates;
- rank using product and session features; and
- return an explainable product list.

Output: a small, ranked product shortlist.

Design boundary: cached product representations make broad retrieval possible; detailed query-product matching waits until the shortlist.



Code walk: a lexical first pass

CANDIDATE GENERATION · LEXICAL OVERLAP

```
# query + session terms
query_terms = {"compact", "carry-on", "luggage",
               "cabin", "travel"}

# score one product
def lexical_score(product):
    # count overlapping index terms
    return len(query_terms &
product["index_terms"])

# sort and keep top four
top_k = sorted(catalogue, key=lexical_score,
```

K_1 · PLAUSIBLE PRODUCT CANDIDATES

Score	Product	Matching index terms
3	SKU-LUG-204 Compact Carry-on Spinner	compact · carry-on · luggage
1	SKU-LUG-118 Hard-shell Carry-on	carry-on
1	SKU-BPK-619 40L Cabin Backpack	cabin
1	SKU-TOT-730 Travel Tote	travel

The terms come from Maya's query plus observable session language, such as the cabin-bag comparison.

Other low-score matches in K_1 · lexical distractors

SKU-ACC-404 · Compact travel umbrella *(compact, travel)*

SKU-ACC-277 · Carry-on luggage scale *(carry-on, luggage)*

SKU-ACC-912 · Cabin packing cubes *(cabin)*

SKU-ACC-056 · Travel neck pillow *(travel)*

Interpret the score as retrieval evidence, not a recommendation score. The next stage can inspect price, dimensions, delivery, and richer query-product fit.



A product candidate needs evidence for later ranking

CATALOGUE → CANDIDATES

Retrieve a product at all

Carry product ID, title, category, and cached representation.

CANDIDATES → RANKER

Compare practical fit

Make price, attributes, popularity, and session signals available.

RANKER → RE-RANKER

Reserve a shortlist

Pass only products worth richer query-product comparison.

RE-RANKER → MAYA

Explain the shortlist

Return product reasons, comparable alternatives, and near-duplicate control.

Failure to audit: if SKU-LUG-204 never enters the candidate set, no later ranker can recommend it.



When Google results danced

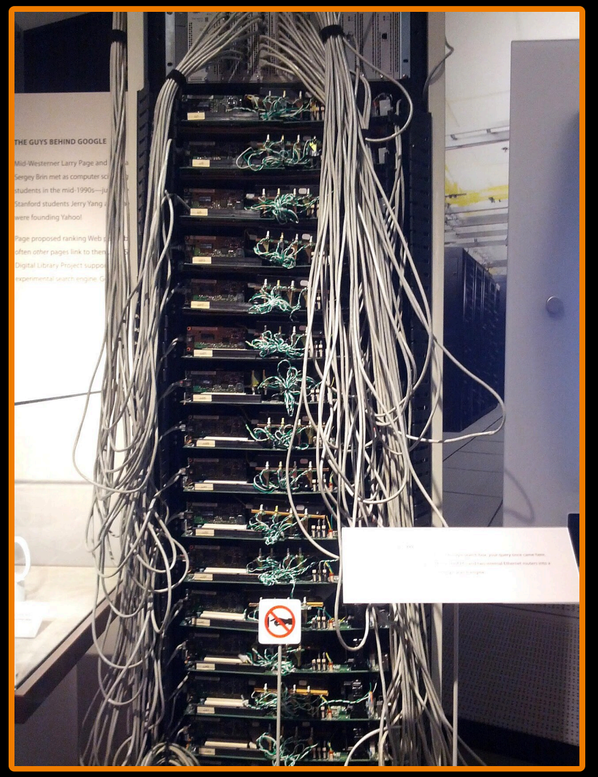
WHILE YOU PAUSE · A SEARCH-HISTORY ASIDE

In 2003, the *Google Dance* described a monthly index update. For several days, searches could alternate between old and new index versions—so rankings appeared to move.

Read the contemporaneous 2003 account ↗

05:00

▶ Start Timer



One of Google's first production server racks (1998).
Photographing Travis · CC BY 2.0 · Wikimedia Commons ↗

03 Retrieval Architectures



Keywords help—until the catalogue uses different words

MAYA'S CURRENT QUERY

"compact carry-on luggage for a five-day trip"

Exact terms make the carry-on spinners easy to find.

> **Lexical strength:** product codes, dimensions, brands, and exact category terms.

CATALOGUE WORDING

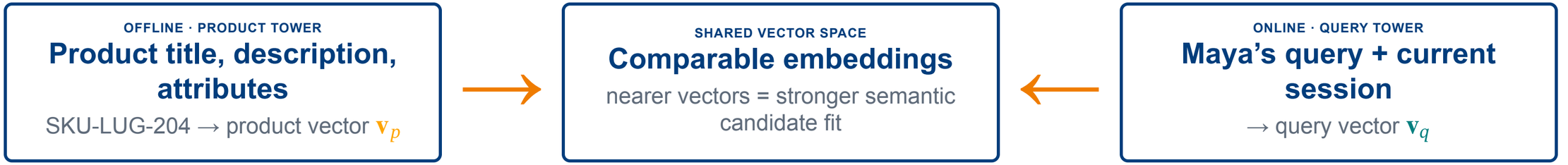
SKU-BPK-619 · "40L Cabin Backpack"

It may suit a five-day trip even though its title contains neither "compact" nor "luggage."

> **Retrieval question:** can the system preserve semantic matches without losing exact constraints?

Candidate generation needs complementary signals: lexical matching for exact language; dense matching for related meaning.

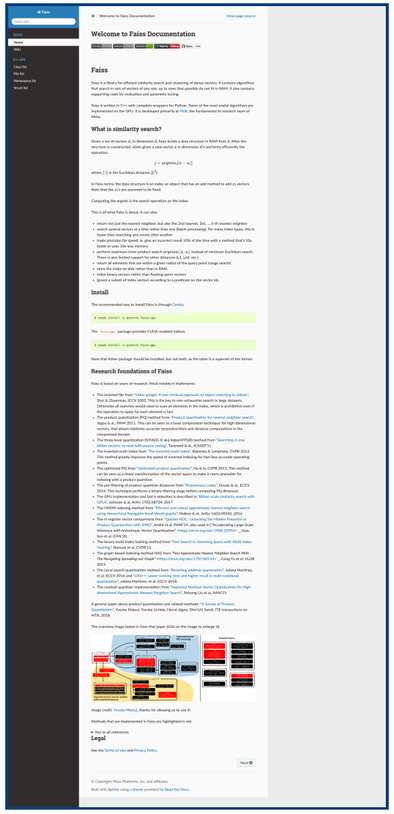
Two towers create a shared comparison space



Training: historical sessions and observed product interactions teach the towers which query-product pairs belong near each other.

Retrieval: compare $\mathbf{v}_q$ with cached $\mathbf{v}_p$ vectors; do not run the product tower again for every query.

FAISS: a library for dense-vector search



faiss.ai ↗ · efficient similarity search and clustering of dense vectors

Offline: build the dense product index

AFTER TRAINING THE TOWERS

```
product_vectors = product_tower.encode(products)
# product → vector
faiss.normalize_L2(product_vectors) # IP = cosine

base = faiss.IndexFlatIP(d) # d = 768; exact IP
index = faiss.IndexIDMap(base) # retain SKU IDs
index.add_with_ids(product_vectors, sku_ids) #
store vectors + SKUs
```

WHAT THE INDEX RETAINS

Vector: searchable representation of each product.

SKU ID: stable pointer back to title, price, image, and inventory metadata.

Refresh path: new or materially changed products need a new vector and index entry.

The two-tower model creates vectors. FAISS stores and searches them.

Normalised inner product: with L2-normalised vectors, `IndexFlatIP` gives an exact cosine-similarity baseline.



Exact dense search is the baseline to beat

EXACT `INDEXFLATIP` SEARCH

Compare Maya’s vector with every product vector

PER-QUERY WORK

O(Nd)

5,000,000 products × 768 dimensions ≈ 3.84 billion dimension-wise comparisons

Exact search is often the correct baseline for a smaller catalogue or an offline audit.

RAW VECTOR STORAGE

Float32 vectors occupy real memory

VECTOR STORE

≈ 15.4 GB

5,000,000 × 768 × 4 bytes; before SKU IDs, metadata, and index overhead

The catalogue scale—not the presence of embeddings—motivates approximate search.



FAISS/HNSW: traverse promising product neighbours

GRAPH ANN CODE WALK

```
base = faiss.IndexHNSWFlat(d, 32)
index = faiss.IndexIDMap(base)
index.add_with_ids(product_vectors, sku_ids)

base.hnsw.efSearch = 64
scores, ids = index.search(maya_vector, k=100)
```

HNSW TRADE-OFF

Explore a graph, not every vector

M = 32: more graph links can improve recall but consume more memory.

efSearch = 64: more exploration usually improves recall but costs more time.

Storage: raw vectors still need ≈15.4 GB; 32 base-layer links add roughly 1.3 GB before overhead.

Candidate contract: HNSW may return fewer comparisons and lower latency,



IVF: search selected vector clusters

INVERTED-FILE (IVF) INTUITION

Cluster product vectors before queries arrive

5M vectors
offline k-means

→

4,096 lists
coarse clusters

→

Probe 32 lists
search a fraction

AVERAGE-WORK ESTIMATE

`nprobe` is a recall–latency control

APPROXIMATE VECTORS SCANNED

≈ 39,000

5,000,000 × 32 / 4,096 ≈ 1/128 of a full scan, before coarse assignment

`IndexIVFFlat` keeps full vectors; IVF reduces search work, not raw-vector storage. IVF-PQ can reduce storage at additional approximation cost.



Bandit Time: What does `nprobe` change?

Maya’s five-million-product index has 4,096 IVF lists. The system raises `nprobe` from 32 to 128. What is the most defensible expectation?

A. The vectors are automatically compressed, so storage falls.

B. More lists are searched, so recall may rise and query work usually rises.

C. The ranker can recover every product omitted by IVF.

D. The two-tower model no longer needs product vectors.

00:45

▶ Start Timer

👥 Retrieval design clinic: choose the candidate index

Your group owns one deployment scenario. Choose the initial candidate index and defend the trade-off.

A · audit baseline

5M vectors; exact reproducibility matters; offline or relaxed latency.

B · live shopping

5M vectors; high Recall@K_1; raw vectors fit in memory; tight latency.

C · marketplace scale

100M vectors; raw-vector storage is costly; bounded query budget.

03:00

▶ Start Timer

Choose **Flat, HNSW, or IVF / IVF-PQ**. Report: (1) the index; (2) one recall–latency or storage consequence; and (3) the exact-baseline comparison you would run.



Candidate generation is a measured contract

HYBRID CANDIDATE SCORE

Protect exact terms and semantic neighbours

$$s_{candidate}(p) = \alpha \cdot s_{lexical}(p) + (1 - \alpha) \cdot s_{dense}(p)$$

Lexical signals preserve exact dimensions, brands, and product codes. Dense signals preserve meaning such as “Cabin Backpack.”

EVALUATE BEFORE RANKING

Measure candidate quality at K_1

Recall@K_1: did suitable products survive?

Latency: can retrieval meet the request budget?

Storage: do vectors and index structure fit the deployment?

Freshness: do changed products receive updated representations?

Bridge to ranking: FAISS and hybrid retrieval preserve broad candidates. The ranker and re-ranker decide which surviving products deserve the top positions.

04 Ranking, Re-ranking, and Visibility

Ranking allocates attention

WORKED REQUEST

Amir seeks a Singapore product-analytics internship

He can work in Singapore, wants a role starting in May, values analytics work over generic marketing, and needs an application deadline he can still meet.

Illustrative candidates and scores in this section are invented for teaching.

THE VISIBLE SURFACE

Six retrieved jobs; three first-page positions



Top positions concentrate attention.
The final order therefore allocates visibility as well as predicted relevance.

Code walk: filter first, score second

1 · Eligibility and lightweight score

```
# illustrative pandas-style walkthrough
eligible = jobs.query(
  "singapore and authorised and days_to_close >= 7"
)

eligible["base_score"] = (
  .50 * eligible.skills_match
  + .30 * eligible.role_intent
  + .20 * eligible.recency
)
shortlist = eligible.nlargest(4, "base_score")
```

The weights are a policy choice, not facts discovered by the code.

2 · Inspect the intermediate output

Role	Skills	Intent	Recent	Score
MarinaLab · product analyst	0.86	0.95	0.80	**0.876**
Northstar · data intern	0.91	0.74	0.90	0.857
CivicPulse · insights intern	0.79	0.88	0.85	0.826
AtlasRetail · analytics intern	0.82	0.77	0.75	0.779

Question: what important preference or protection is still absent from this score?



♠ Spades: keep important attributes fixed

WITHOUT A STABLE CONSTRAINT

A high score can still produce the wrong recommendation

> **Northstar · data intern**
>
> 0.857 score — but the role requires a work authorisation Amir does not hold.

A model should not ask a user to infer that a critical requirement was ignored.

♠ LOYALIST / METADATA-DRIVEN LENS

Filters, consistency, and explanation

- **Filter:** Singapore, eligibility, deadline, role type.
- **Keep fixed:** an explicit constraint remains true after re-ranking.
- **Explain:** “Shown because it matches your analytics coursework and May start preference.”

🤔 **To think about**

Which attribute should be an inviolable filter here, and which should remain a tradeable ranking feature?

Code walk: re-rank a shortlist, not the whole corpus

A richer list-level pass

```
# only after retrieval + lightweight ranking
final = rerank(
  shortlist,
  query="product analytics internship",
  must_keep=["eligible", "singapore"],
  max_per_employer=1,
  freshness_bonus=0.04
)

display(final[["rank", "role", "why"]])
```

The exact method can vary; its list-level constraints must be explicit and testable.

Final visible list

1. **MarinaLab · product analyst**

   strong fit; eligible; still open

2. **CivicPulse · insights intern**

   strong fit; recent listing; distinct employer

3. **AtlasRetail · analytics intern**

   eligible; preserves a qualified retail-analytics option

Re-ranking changes an ordered list, not the set of basic eligibility rules.



♦ Diamonds: retain qualified novelty and exposure

BEFORE LIST-LEVEL INTERVENTION

Relevant—but concentrated

1
MarinaLab · product analyst

2
MarinaLab · data intern

3
Northstar · data intern

Repeated employers and familiar items can crowd out a qualified new option.

♦ NOVELTY-SEEKER LENS

Freshness, diversity, serendipity, new-item exposure

1
MarinaLab · product analyst

2
CivicPulse · insights intern
new employer

3
AtlasRetail · analytics intern
recent

Novelty is constrained by qualification; it is not random exploration.

♥ Hearts: social evidence can help and distort

A TEMPTING FEATURE

“47 students from your programme saved this role”

This cue may reduce uncertainty. It may also disclose a peer group’s behaviour, trigger conformity, or make a weak job appear safer than it is.

DESIGN QUESTION

Show?

To whom, with what threshold, and with what explanation?

♥ SOCIAL LENS

Social evidence, influence, and conformity risk

- Aggregate and threshold a cue before display.
- Do not imply a peer endorsement the data cannot support.
- Test whether the cue shifts choices toward popularity rather than fit.
- Give the user a way to hide social signals.

♣ Clubs: protect the list from manipulation

SUSPICIOUS SIGNAL

A weak listing rises after a burst of saves

Role	Fit	Saves	Concern
BrightPath · analyst trainee	0.41	1,240 / 2 h	**coordinated burst**

If raw engagement enters the ranker, a coordinated campaign can buy visibility.

♣ ADVERSARIAL / SHILL LENS

Manipulation, coordinated abuse, robustness

```
if suspicious_save_burst(job):
    job.engagement_weight = 0
    job.review_state = "hold"

ranker.fit_score(job)
# preserve an audit trail; do not
silently erase it
```

Robustness needs monitoring and investigation, not a permanent assumption that engagement is trustworthy.



👥 Audit one internship list through four treatment lenses

In groups, audit the three-role final list. For each Week 01 card-suit lens, name one check or safeguard the system needs.

♠ Spades

Which stable attribute must remain fixed, and how will the user see why?

♦ Diamonds

How will you check qualified freshness, diversity, or new-item exposure?

♥ Hearts

What social cue needs a privacy or conformity safeguard?

♣ Clubs

Which signal could be manipulated, and what robustness check responds?

03:00

▶ Start Timer

Prepare a four-line audit: one check per suit. Be ready to justify one choice from the final ranked list.



🔑 Re-ranking makes trade-offs inspectable

Retrieve plausible options → **filter** non-negotiables → **rank** lightweight fit → **re-rank** the shortlist → **audit** the visible list.

The engineering claim

Reserve richer, list-level computation for a small candidate set—and make its constraints explicit.

The treatment-lens claim

A good list protects stable constraints, qualified novelty, careful social evidence, and robustness against manipulation.

Summary



🔑 Pipeline stages allocate both computation and visibility.

A candidate missing from K_1 cannot be rescued later. A surviving candidate still has to earn a visible position.

PRESERVE POSSIBILITIES

Maya's candidate contract · $N \rightarrow K_1$

Use lexical, dense, or hybrid evidence to retain plausible products before expensive judgement.

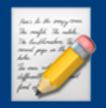
Measure: Recall@K_1 · latency · storage · freshness

ALLOCATE ATTENTION

Shortlist contract · $K_1 \rightarrow K_2 \rightarrow K_3$

Use richer evidence only for a smaller set, then make the final order and its protections inspectable.

Inspect: constraints · explanation · exposure · robustness



Make one defensible deployment claim

Write one three-line claim for a recommendation surface you know: a shopping, course-resource, media, or internship list.

1 · Preserve: “For *[surface]*, retrieve K_1 using *[candidate evidence]* and measure *[recall / latency / freshness]*.”

2 · Allocate: “Rank or re-rank using *[richer evidence]*, while keeping *[one hard constraint or explanation]* visible.”

3 · Protect: choose one Week 01 treatment lens: ♠ stable attributes / explanation · ♦ novelty / diversity · ♥ social influence / conformity · ♣ shills / robustness.

01:30

▶ Start Timer

Write for 90 seconds, then compare one line with a neighbour. Be ready to share the failure check—not just the model choice.





NEXT WEEK PREVIEW — WEEK 07

Quiz and Project Design Critique Workshop

Mitch Altman · CC BY-SA 2.0 · Wikimedia Commons ↗